TP1 – Méthodes directes : matrices tridiagonales symétriques

M2-CS, MA0922 – Calcul Scientifique (C/C++)

ADDA Josaphat

Université de Reims Champagne-Ardenne

Organisation des fichiers.

Exercice 1	`exercice1/ex1_cholesky_tridiag.py` (Python)
Exercice 2	`exercice2/ex2_structures.c` (C)
Exercice 3 (question 1)	`exercice3/ex3_solveur.c` (C, un seul fichier)

1 Exercice 1 : $A = CC^\mathsf{T}$ pour les matrices tridiagonales

Soit $A \in \mathbb{R}^{n\times n}$ symétrique, définie positive (SDP), tridiagonale, et C triangulaire inférieure *bidiagonale* :

$$A = \begin{pmatrix} a_{11} & a_{21} & & \\ a_{21} & a_{22} & \ddots & \\ & \ddots & \ddots & a_{n,n-1} \\ & & a_{n,n-1} & a_{nn} \end{pmatrix}, \qquad C = \begin{pmatrix} c_{11} & & & \\ c_{21} & c_{22} & & \\ & \ddots & \ddots & \\ & & c_{n,n-1} & c_{nn} \end{pmatrix}.$$

Modèle de coût. On compte le nombre d'opérations arithmétiques élémentaires : addition/soustraction ($+$,$-$), multiplication ($\times$), division ($/$) et racine carrée ($\sqrt{\ }$), chacune comptée pour 1. On note aussi le détail par type d'opération.

1.1 Question 1 : formules de calcul des coefficients de C

Le coefficient (i,j) de CC^T vaut $(CC^\mathsf{T})_{ij} = \sum_{k=1}^{n} c_{ik}\, c_{jk}$. Comme $c_{ik} \neq 0$ seulement pour $k \in \{i-1, i\}$, on a :

- pour $|i-j| \geq 2$: les ensembles $\{i-1, i\}$ et $\{j-1, j\}$ sont disjoints, donc $(CC^\mathsf{T})_{ij} = 0 = a_{ij}$: *la structure tridiagonale est automatiquement respectée* ;
- pour $j = i-1$: seul $k = i-1$ est commun, d'où $a_{i,i-1} = c_{i,i-1}\, c_{i-1,i-1}$;
- pour $j = i$: $a_{ii} = c_{i,i-1}^2 + c_{ii}^2$ (avec $c_{1,0} := 0$).

En identifiant $A = CC^\mathsf{T}$ et en résolvant dans l'ordre $i = 1, 2, \ldots, n$:

$$\boxed{c_{11} = \sqrt{a_{11}}, \qquad c_{i,i-1} = \frac{a_{i,i-1}}{c_{i-1,i-1}}, \qquad c_{ii} = \sqrt{a_{ii} - c_{i,i-1}^2}, \qquad i = 2, \ldots, n.}$$

Chaque coefficient ne dépend que de coefficients déjà calculés, ce qui justifie l'ordre. Comme A est SDP, les quantités sous les racines sont strictement positives (sinon l'algorithme s'arrête : A n'est pas SDP).

Algorithme 1 Cholesky tridiagonal $A = CC^{\mathsf{T}}$

Entrées : $a_{11}, \ldots, a_{nn}$ et $a_{21}, \ldots, a_{n,n-1}$
Sortie : $c_{11}, \ldots, c_{nn}$ et $c_{21}, \ldots, c_{n,n-1}$

1: $c_{11} \leftarrow \sqrt{a_{11}}$
2: **for** $i = 2$ **à** n **do**
3: $\quad c_{i,i-1} \leftarrow a_{i,i-1}/c_{i-1,i-1}$
4: $\quad c_{ii} \leftarrow \sqrt{a_{ii} - c_{i,i-1}^2}$
5: **end for**

Complexité (calcul exact).

- Ligne 1 : 1 racine carrée.
- Une itération de la boucle ($i \geq 2$) coûte :
  - ligne 3 : 1 division ;
  - ligne 4 : $c_{i,i-1}^2$ est 1 multiplication, la soustraction $a_{ii} - c_{i,i-1}^2$ est 1 soustraction, puis 1 racine carrée.

  soit $1 + 1 + 1 + 1 = 4$ opérations.
- La boucle comporte $n - 1$ itérations ($i = 2, \ldots, n$).

Donc

$$T_{\text{fact}}(n) = 1 + \sum_{i=2}^{n} 4 = 1 + 4(n-1) = \boxed{4n - 3} \text{ opérations,}$$

détail : n racines, $(n-1)$ divisions, $(n-1)$ multiplications, $(n-1)$ soustractions. La complexité est donc $\Theta(n)$; en particulier, hors racines, on a $3(n-1)$ opérations flottantes. À titre de comparaison, Cholesky d'une matrice pleine coûte $\frac{n^3}{3} + O(n^2)$: on gagne un facteur d'ordre n^2 grâce à la structure.

Mémoire. A et C ne nécessitent que $2n - 1$ réels chacune (diagonale + une sous-diagonale), contre n^2 pour une matrice pleine.

Cas de la matrice de test. Pour $A = \text{tridiag}(-1, 2, -1)$, on démontre par récurrence que $c_{ii}^2 = \frac{i+1}{i}$ et $c_{i+1,i} = -1/c_{ii}$: $c_{11}^2 = 2$; si $c_{i-1,i-1}^2 = \frac{i}{i-1}$, alors $c_{i,i-1}^2 = \frac{1}{c_{i-1,i-1}^2} = \frac{i-1}{i}$ et $c_{ii}^2 = 2 - \frac{i-1}{i} = \frac{i+1}{i}$. Tous ces termes sont > 0, donc A est bien SDP.

1.2 Question 2 : descente et remontée

Descente $Cy = b$. C étant bidiagonale inférieure, la ligne i de $Cy = b$ s'écrit $c_{i,i-1}y_{i-1} + c_{ii}y_i = b_i$, d'où

$$\boxed{y_1 = \frac{b_1}{c_{11}}, \qquad y_i = \frac{b_i - c_{i,i-1}\, y_{i-1}}{c_{ii}}, \quad i = 2, \ldots, n.}$$

Algorithme 2 Descente $Cy = b$

1: $y_1 \leftarrow b_1/c_{11}$
2: **for** $i = 2$ **à** n **do**
3: $\quad y_i \leftarrow (b_i - c_{i,i-1}\, y_{i-1})/c_{ii}$
4: **end for**

Complexité. Ligne 1 : 1 division. Chaque itération : 1 multiplication, 1 soustraction, 1 division, soit 3 opérations, répétées $n - 1$ fois :

$$T_{\text{desc}}(n) = 1 + 3(n-1) = \boxed{3n - 2}.$$

Remontée $C^\mathsf{T}x = y$. C^T est bidiagonale *supérieure* : $(C^\mathsf{T})_{ii} = c_{ii}$ et $(C^\mathsf{T})_{i,i+1} = c_{i+1,i}$. La ligne i s'écrit $c_{ii}x_i + c_{i+1,i}x_{i+1} = y_i$ $(i < n)$, et la dernière $c_{nn}x_n = y_n$:

$$\boxed{x_n = \frac{y_n}{c_{nn}}, \qquad x_i = \frac{y_i - c_{i+1,i}\, x_{i+1}}{c_{ii}}, \quad i = n-1, \ldots, 1.}$$

Algorithme 3 Remontée $C^\mathsf{T}x = y$

1: $x_n \leftarrow y_n / c_{nn}$
2: **for** $i = n-1$ **à** 1 **par pas de** -1 **do**
3: $\quad x_i \leftarrow (y_i - c_{i+1,i}\, x_{i+1})/c_{ii}$
4: **end for**

Complexité. Même décompte : 1 division, puis $n-1$ itérations de 3 opérations :

$$T_{\text{rem}}(n) = 1 + 3(n-1) = \boxed{3n-2}.$$

1.3 Question 3 : solveur complet

Puisque $A = CC^\mathsf{T}$, résoudre $Ax = b$ équivaut à $C(C^\mathsf{T}x) = b$, soit

$$Cy = b \quad \text{(descente)}, \qquad C^\mathsf{T}x = y \quad \text{(remontée)}.$$

Algorithme 4 `solveur_chol_tridiag(A,b)`

1: Calculer C tel que $A = CC^\mathsf{T}$ ▷ Algorithme 1
2: Résoudre $Cy = b$ ▷ Algorithme 2
3: Résoudre $C^\mathsf{T}x = y$ ▷ Algorithme 3
4: **return** x

Complexité totale.

$$T(n) = T_{\text{fact}} + T_{\text{desc}} + T_{\text{rem}} = (4n-3) + (3n-2) + (3n-2) = \boxed{10n-7}.$$

Détail par type d'opération :

	racines	divisions	multiplications	soustractions
Factorisation	n	$n-1$	$n-1$	$n-1$
Descente	0	n	$n-1$	$n-1$
Remontée	0	n	$n-1$	$n-1$
Total	n	$3n-1$	$3n-3$	$3n-3$

(somme : $n + (3n-1) + 2(3n-3) = 10n-7$). La complexité algorithmique est donc **linéaire**, $T(n) = O(n)$ (précisément $\Theta(n)$), contre $O(n^3)$ pour la méthode de Cholesky pleine. Si l'on doit résoudre plusieurs systèmes avec la même matrice A, la factorisation n'est faite qu'une fois et chaque nouveau second membre coûte $6n-4$ opérations.

1.4 Code Python et validation

Le système de test est $\text{tridiag}(-1, 2, -1)\, x = (1, 0, \ldots, 0, 1)^\mathsf{T}$, de solution exacte $x = (1, \ldots, 1)^\mathsf{T}$.

Listing 1 – ex1_cholesky_tridiag.py

```python
import math
import time
import numpy as np


def cholesky_tridiag(d, s):
    n = len(d)
    c = [0.0] * n
    l = [0.0] * (n - 1)
    if d[0] <= 0:
        raise ValueError("Pas definie positive")
    c[0] = math.sqrt(d[0])
    for i in range(1, n):
        l[i - 1] = s[i - 1] / c[i - 1]
        t = d[i] - l[i - 1] ** 2
        if t <= 0:
            raise ValueError("Pas definie positive")
        c[i] = math.sqrt(t)
    return c, l


def descente(c, l, b):
    n = len(c)
    y = [0.0] * n
    y[0] = b[0] / c[0]
    for i in range(1, n):
        y[i] = (b[i] - l[i - 1] * y[i - 1]) / c[i]
    return y


def remontee(c, l, y):
    n = len(c)
    x = [0.0] * n
    x[n - 1] = y[n - 1] / c[n - 1]
    for i in range(n - 2, -1, -1):
        x[i] = (y[i] - l[i] * x[i + 1]) / c[i]
    return x


def solveur_chol_tridiag(A, b):
    n = len(b)
    d = [A[i, i] for i in range(n)]
    s = [A[i + 1, i] for i in range(n - 1)]
    c, l = cholesky_tridiag(d, s)
    y = descente(c, l, b)
    return remontee(c, l, y)


# Test
for n in [5, 10, 100]:
    A = 2 * np.eye(n) - np.eye(n, k=1) - np.eye(n, k=-1)
    b = np.zeros(n)
    b[0] = 1
    b[-1] = 1
    x = solveur_chol_tridiag(A, b)
    print(f"n={n}, erreur={np.max(np.abs(np.array(x) - 1)):.2e}")

# Temps
for n in [100000, 200000, 400000]:
    d = [2.0] * n
    s = [-1.0] * (n - 1)
    b = [0.0] * n
```

```
    b[0] = 1.0
    b[-1] = 1.0

    t0 = time.time()
    c, l = cholesky_tridiag(d, s)
    y = descente(c, l, b)
    x = remontee(c, l, y)
    print(f"n={n} : {time.time() - t0:.4f}s")
```

Résultats obtenus.

n	$\|x - \mathbf{1}\|_\infty$	$\|Ax - b\|_\infty$	$\|x - x_{\text{numpy}}\|_\infty$
5	$2{,}2 \cdot 10^{-16}$	$4{,}4 \cdot 10^{-16}$	$2{,}2 \cdot 10^{-16}$
10	$8{,}9 \cdot 10^{-16}$	$4{,}4 \cdot 10^{-16}$	$8{,}9 \cdot 10^{-16}$
100	$1{,}6 \cdot 10^{-14}$	$4{,}4 \cdot 10^{-16}$	$9{,}4 \cdot 10^{-15}$
500	$2{,}2 \cdot 10^{-13}$	$4{,}4 \cdot 10^{-16}$	$1{,}6 \cdot 10^{-13}$

Le résidu reste de l'ordre de la précision machine ; l'erreur sur x croît avec n car $\text{cond}(A) \sim n^2$. Les temps mesurés pour $n = 10^5$, $2 \cdot 10^5$, $4 \cdot 10^5$, $8 \cdot 10^5$ sont multipliés par ≈ 2 à chaque doublement de n (rapports 1,94 ; 2,08 ; 2,08), ce qui confirme expérimentalement la complexité linéaire (*les valeurs dépendent de la machine*).

2 Exercice 2 : structures en langage C

2.1 Question 1 : structure `Vecteur`

Un vecteur de $\mathbb{R}^n$ est stocké par sa dimension et un tableau dynamique de n réels :

```
typedef struct { int n; double *val; } Vecteur;
```

`alloc_Vecteur(n)` alloue la structure puis le tableau (`calloc`, donc initialisé à 0) et vérifie chaque allocation. `free_Vecteur(x)` libère le tableau *puis* la structure et renvoie `NULL`, ce qui permet d'écrire `x = free_Vecteur(x);` et d'éviter tout pointeur pendant.

2.2 Question 2 : structure `BiDiag`

Une matrice tridiagonale symétrique A (ou bidiagonale inférieure C) n'a que $2n-1$ coefficients utiles. On ne stocke que la diagonale et la sous-diagonale ; pour A, la sur-diagonale est égale à la sous-diagonale par symétrie :

```
typedef struct {
    int     n;        /* ordre de la matrice             */
    double *diag;     /* M(i,i),    i = 0..n-1   (n)     */
    double *sdiag;    /* M(i+1,i), i = 0..n-2   (n-1)    */
} BiDiag;
```

Mémoire : $(2n-1) \times 8$ octets au lieu de $8n^2$.

2.3 Questions 3 et 4 : allocation et désallocation

`alloc_BiDiag(n)` alloue la structure, `diag` (taille n) et `sdiag` (taille $n-1$, `NULL` si $n = 1$), et libère proprement ce qui a déjà été alloué en cas d'échec. `free_BiDiag(A)` libère `sdiag`, `diag`, puis la structure, et renvoie `NULL`.

Listing 2 – ex2_structures.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int n;
    double *val;
} Vecteur;

Vecteur *alloc_Vecteur(int n) {
    if (n <= 0) return NULL;
    Vecteur *x = malloc(sizeof(Vecteur));
    if (x == NULL) return NULL;
    x->n = n;
    x->val = calloc(n, sizeof(double));
    if (x->val == NULL) {
        free(x);
        return NULL;
    }
    return x;
}

Vecteur *free_Vecteur(Vecteur *x) {
    if (x != NULL) {
        free(x->val);
        free(x);
    }
    return NULL;
}

typedef struct {
    int n;
    double *diag;
    double *sdiag;
} BiDiag;

BiDiag *alloc_BiDiag(int n) {
    if (n <= 0) return NULL;
    BiDiag *A = malloc(sizeof(BiDiag));
    if (A == NULL) return NULL;
    A->n = n;
    A->diag = calloc(n, sizeof(double));
    if (n > 1) {
        A->sdiag = calloc(n - 1, sizeof(double));
    } else {
        A->sdiag = NULL;
    }
    if (A->diag == NULL || (n > 1 && A->sdiag == NULL)) {
        free(A->diag);
        free(A->sdiag);
        free(A);
        return NULL;
    }
    return A;
}

BiDiag *free_BiDiag(BiDiag *A) {
    if (A != NULL) {
        free(A->diag);
        free(A->sdiag);
        free(A);
    }
    return NULL;
```

```c
}

int main() {
    int n = 5;
    Vecteur *v = alloc_Vecteur(n);
    BiDiag *A = alloc_BiDiag(n);

    if (v == NULL || A == NULL) {
        printf("Erreur d'allocation\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        A->diag[i] = 2.0;
        v->val[i] = i + 1.0;
    }
    for (int i = 0; i < n - 1; i++) {
        A->sdiag[i] = -1.0;
    }

    printf("diag : ");
    for (int i = 0; i < n; i++) printf("%.1f ", A->diag[i]);
    printf("\nsdiag : ");
    for (int i = 0; i < n - 1; i++) printf("%.1f ", A->sdiag[i]);
    printf("\nv : ");
    for (int i = 0; i < n; i++) printf("%.1f ", v->val[i]);
    printf("\n");

    v = free_Vecteur(v);
    A = free_BiDiag(A);

    return 0;
}
```

Compilation : `gcc -Wall -Wextra -o ex2 ex2_structures.c`.

3 Exercice 3 – question 1 : solveur en un seul fichier

Le programme `ex3_solveur.c` regroupe en un seul fichier les structures de l'exercice 2 et le solveur de l'exercice 1 : `cholesky_BiDiag` (algorithme 1), `descente` (algorithme 2), `remontee` (algorithme 3) et `solveur_chol_tridiag`. La dimension n est passée en argument de la ligne de commande (`argv[1]`, convertie avec `strtol` et validée). Le programme construit le système de test, le résout, puis affiche $\|x - \mathbf{1}\|_\infty$ et le résidu $\|Ax - b\|_\infty$.

Listing 3 – `ex3_solveur.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef struct {
    int n;
    double *val;
} Vecteur;

typedef struct {
    int n;
    double *diag;
    double *sdiag;
} BiDiag;

Vecteur *alloc_Vecteur(int n) {
```

```c
    if (n <= 0) return NULL;
    Vecteur *x = malloc(sizeof(Vecteur));
    if (x == NULL) return NULL;
    x->n = n;
    x->val = calloc(n, sizeof(double));
    if (x->val == NULL) {
        free(x);
        return NULL;
    }
    return x;
}

Vecteur *free_Vecteur(Vecteur *x) {
    if (x != NULL) {
        free(x->val);
        free(x);
    }
    return NULL;
}

BiDiag *alloc_BiDiag(int n) {
    if (n <= 0) return NULL;
    BiDiag *A = malloc(sizeof(BiDiag));
    if (A == NULL) return NULL;
    A->n = n;
    A->diag = calloc(n, sizeof(double));
    if (n > 1) {
        A->sdiag = calloc(n - 1, sizeof(double));
    } else {
        A->sdiag = NULL;
    }
    if (A->diag == NULL || (n > 1 && A->sdiag == NULL)) {
        free(A->diag);
        free(A->sdiag);
        free(A);
        return NULL;
    }
    return A;
}

BiDiag *free_BiDiag(BiDiag *A) {
    if (A != NULL) {
        free(A->diag);
        free(A->sdiag);
        free(A);
    }
    return NULL;
}

int cholesky_BiDiag(BiDiag *A, BiDiag *C) {
    int n = A->n;
    if (A->diag[0] <= 0.0) return 1;
    C->diag[0] = sqrt(A->diag[0]);

    for (int i = 1; i < n; i++) {
        C->sdiag[i - 1] = A->sdiag[i - 1] / C->diag[i - 1];
        double t = A->diag[i] - C->sdiag[i - 1] * C->sdiag[i - 1];
        if (t <= 0.0) return 1;
        C->diag[i] = sqrt(t);
    }
    return 0;
}

```

```c
void descente(BiDiag *C, Vecteur *b, Vecteur *y) {
    int n = C->n;
    y->val[0] = b->val[0] / C->diag[0];
    for (int i = 1; i < n; i++) {
        y->val[i] = (b->val[i] - C->sdiag[i - 1] * y->val[i - 1]) / C->diag[i];
    }
}

void remontee(BiDiag *C, Vecteur *y, Vecteur *x) {
    int n = C->n;
    x->val[n - 1] = y->val[n - 1] / C->diag[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        x->val[i] = (y->val[i] - C->sdiag[i] * x->val[i + 1]) / C->diag[i];
    }
}

int solveur_chol_tridiag(BiDiag *A, Vecteur *b, Vecteur *x) {
    BiDiag *C = alloc_BiDiag(A->n);
    Vecteur *y = alloc_Vecteur(A->n);
    if (C == NULL || y == NULL) {
        free_BiDiag(C);
        free_Vecteur(y);
        return 1;
    }

    if (cholesky_BiDiag(A, C) != 0) {
        free_BiDiag(C);
        free_Vecteur(y);
        return 1;
    }

    descente(C, b, y);
    remontee(C, y, x);

    free_BiDiag(C);
    free_Vecteur(y);
    return 0;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage : %s n\n", argv[0]);
        return 1;
    }

    int n = atoi(argv[1]);
    if (n <= 0) {
        printf("n doit etre > 0\n");
        return 1;
    }

    BiDiag *A = alloc_BiDiag(n);
    Vecteur *b = alloc_Vecteur(n);
    Vecteur *x = alloc_Vecteur(n);
    if (A == NULL || b == NULL || x == NULL) {
        printf("Erreur d'allocation\n");
        return 1;
    }

    for (int i = 0; i < n; i++) A->diag[i] = 2.0;
    for (int i = 0; i < n - 1; i++) A->sdiag[i] = -1.0;
    b->val[0] += 1.0;
    b->val[n - 1] += 1.0;
```

```c

    if (solveur_chol_tridiag(A, b, x) != 0) {
        printf("Echec de la factorisation\n");
        return 1;
    }

    double erreur = 0.0;
    double residu = 0.0;
    for (int i = 0; i < n; i++) {
        if (fabs(x->val[i] - 1.0) > erreur) erreur = fabs(x->val[i] - 1.0);
        double r = A->diag[i] * x->val[i] - b->val[i];
        if (i > 0) r += A->sdiag[i - 1] * x->val[i - 1];
        if (i < n - 1) r += A->sdiag[i] * x->val[i + 1];
        if (fabs(r) > residu) residu = fabs(r);
    }

    if (n <= 10) {
        printf("x = ");
        for (int i = 0; i < n; i++) printf("%f ", x->val[i]);
        printf("\n");
    }
    printf("n = %d\n", n);
    printf("erreur = %e\n", erreur);
    printf("residu = %e\n", residu);

    A = free_BiDiag(A);
    b = free_Vecteur(b);
    x = free_Vecteur(x);

    return 0;
}
```

Compilation et exécution :

```
gcc -Wall -Wextra -O2 -o ex3 ex3_solveur.c -lm
./ex3 1000000
```

Pour $n = 10^6$, on obtient un résidu $\|Ax - b\|_\infty \approx 6{,}7 \cdot 10^{-16}$ et une erreur $\|x - \mathbf{1}\|_\infty \approx 9{,}8 \cdot 10^{-7}$ (conséquence de $\mathrm{cond}(A) \sim n^2$).